

Programmazione di Applicazioni Intelligenti

a.a. 2025/2026

daniele.mantovani@uniupo.it



by Daniele Mantovani





Chi sono



Esperienza

30 anni di carriera, dall'ingegneria del software classica alle soluzioni di Intelligenza Artificiale Generativa.



Docente Accademico

23 anni di esperienza di insegnamento presso l'Università del Piemonte Orientale.



Missione Didattica

Costruire un ponte tra teoria accademica e pratiche del mondo del lavoro.

Cosa Impareremo

Questo corso vi accompagnerà in un viaggio completo attraverso l'intelligenza artificiale generativa, dalle fondamenta matematiche fino alla produzione di sistemi reali.



Fondamenta Teoriche

Comprendere la matematica e le architetture dietro l'AI Generativa: Transformer, meccanismi di Attention, e i principi che governano i Large Language Models.



Costruzione Agenti

Creare Agenti Autonomi capaci di ragionare, pianificare e utilizzare strumenti esterni attraverso il Framework Agno e tecniche di orchestrazione avanzate.



Produzione Enterprise

Portare i modelli nel mondo reale: implementazione di RAG, strategie di Fine-Tuning, Deployment scalabile e valutazione sistematica delle performance.



Quando e Dove

1

Calendario Lezioni

Ogni **Mercoledì**, dalle **13:30 alle 17:30**. Un appuntamento fisso di 4 ore intensive per massimizzare l'apprendimento pratico e teorico.

2

Sede del Corso

Aula Lab 1 (salvo diversa comunicazione).

3

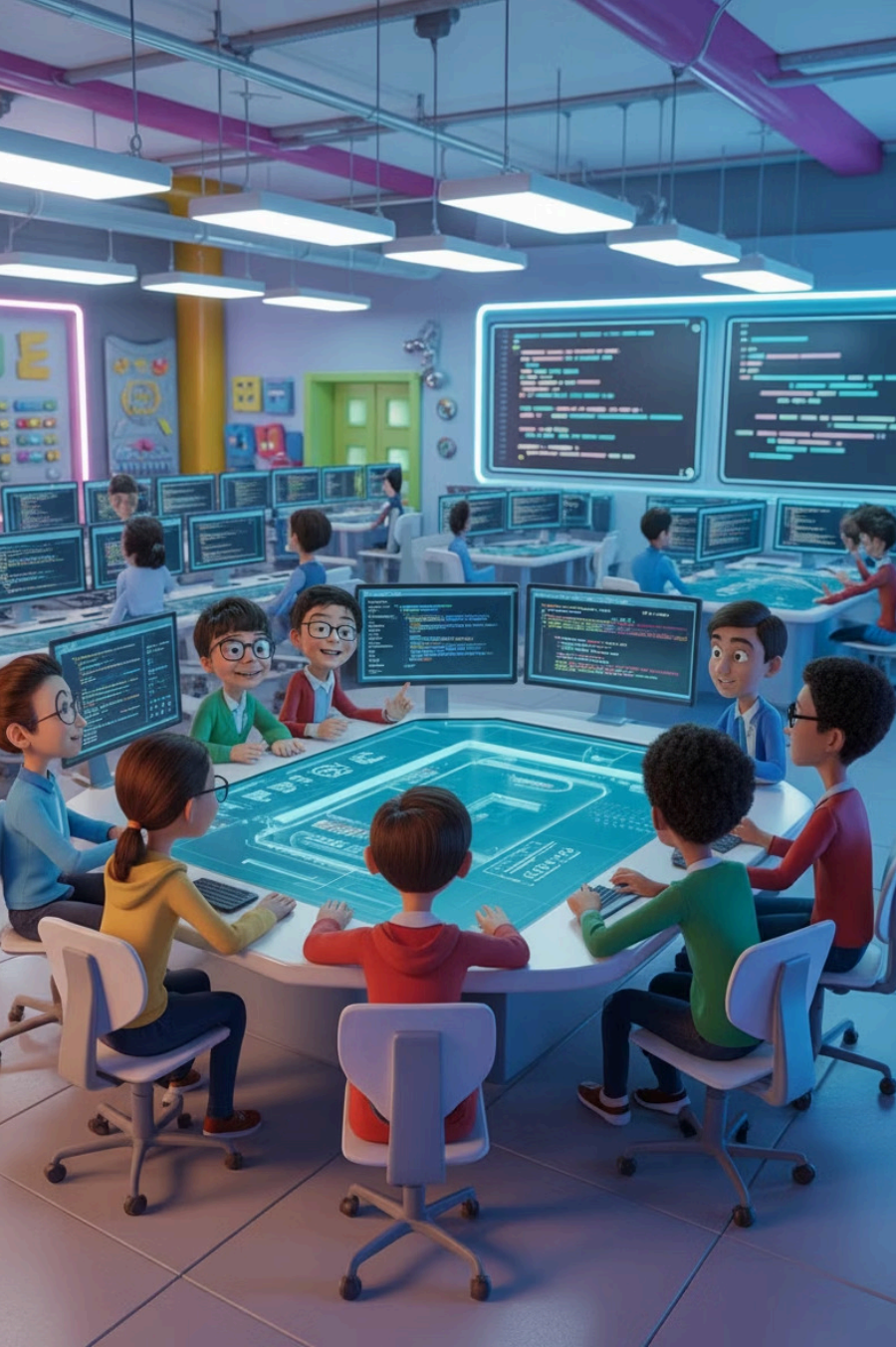
Struttura Temporale

48 ore totali distribuite in **12 lezioni** da 4 ore ciascuna. Un percorso intensivo progettato per bilanciare teoria e pratica.

4

Canali Comunicazione

Moodle (DIR) per materiale e comunicazioni



Come Lavoreremo

Il corso adotta un approccio bilanciato che integra rigore teorico e applicazione pratica, garantendo una comprensione profonda e competenze immediatamente spendibili.



Didattica Frontale

Circa 36 ore dedicate a:

- Fondamenti teorici e matematici degli LLM
- Analisi architetturale dei modelli Transformer
- Discussione di paper scientifici recenti
- Strategie di progettazione di sistemi intelligenti
- Best practices per deployment e produzione



Laboratorio Guidato

Circa 12 ore "hands-on":

- Sessioni di coding live con il docente
- Debug assistito e risoluzione problemi
- Sperimentazione con framework e librerie

Prerequisiti Tecnici

Per affrontare efficacemente il corso, è essenziale possedere alcune competenze di base. Verificate di soddisfare questi requisiti prima dell'inizio.



Python Solido

Conoscenza approfondita di Python: programmazione orientata agli oggetti (classi, ereditarietà), gestione di virtual environments, installazione e configurazione pacchetti con pip/conda/uv.



Matematica Fondamentale

Algebra Lineare di base: operazioni con vettori e matrici, dot product, trasformazioni lineari. Concetti di calcolo differenziale e probabilità saranno utili ma non strettamente necessari.



Version Control

Padronanza di Git per il versionamento del codice: commit, branch, merge, push/pull. Fondamentale per collaborare sul progetto e gestire le iterazioni di sviluppo.

Roadmap del Corso

Il programma è strutturato in quattro moduli progressivi che vi condurranno dalle basi teoriche fino alla realizzazione di applicazioni intelligenti avanzate.



Modulo 1: Fondamenta & Architetture

Lezioni 1-4 | Storia dell'AI, Reti Neurali, Architettura Transformer, Meccanismi di Attention e Self-Attention, Tokenizzazione.



Modulo 2: Prompting & Agenti Base

Lezioni 5-6 | Tecniche di Prompt Engineering, Chain-of-Thought, Framework Agno, Primi agenti autonomi e orchestrazione semplice.



Modulo 3: RAG & Fine-Tuning

Lezioni 7-9 | Retrieval-Augmented Generation, Vector Databases, Embeddings, Parameter-Efficient Fine-Tuning (LoRA, QLoRA).

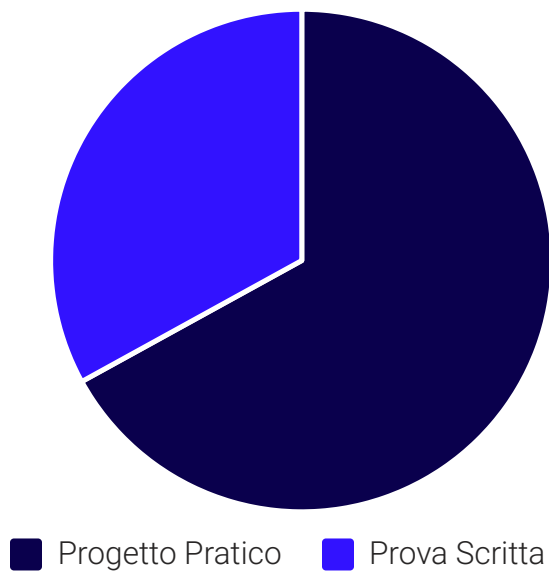


Modulo 4: Advanced Agents & Project

Lezioni 10-12 | Agenti multi-step, Tool Use, Sandboxing, Valutazione e Metriche.

Modalità d'Esame e Valutazione

L'obiettivo dell'esame è valutare sia la vostra capacità ingegneristica di costruire sistemi AI, sia la comprensione teorica dei modelli.



Il Progetto (2/3 del voto finale): Consiste nello sviluppo di un'applicazione intelligente completa, dalla pipeline dati all'interfaccia utente. Contribuisce per i 2/3 del voto finale.

La Prova Scritta (1/3 del voto finale): Verificherà le competenze acquisite sulla teoria e i concetti fondamentali trattati nel corso. Contribuisce per 1/3 del voto finale.

⚠ E' consentito l'uso di assistenza AI nello sviluppo del progetto. Tuttavia, l'incapacità di spiegare il funzionamento del proprio codice (anche se generato) impatterà negativamente sulla valutazione finale.



Il progetto

L'esame è incentrato sullo sviluppo di un **progetto pratico completo** che determina i **2/3 del voto finale**. L'obiettivo è dimostrare competenze ingegneristiche concrete: dal prompt engineering all'architettura del software.

- ❑ La conoscenza teorica sottostante sarà verificata separatamente nella prova scritta.

Discussione Orale

Walkthrough approfondito del progetto, discussione delle scelte architettureali, motivazione delle decisioni tecniche e analisi critica dei risultati ottenuti.

Lavoro di Gruppo

Possibilità di lavorare in team (max 3 persone).

Framework Consigliati

Agno per gli agenti. Alternative motivate come LangChain o LlamaIndex sono ammesse se giustificate.

Rubrica di Valutazione del Progetto

Criteri di valutazione per il progetto finale del corso. Il progetto contribuisce per i **2/3 del voto finale**. Il punteggio base massimo è **30 punti**, con +2 punti bonus per complessità tecnica e originalità (32 pt totali, con lode).

Architettura e Qualità Tecnica – 8 pt

- Solidità strutturale del codice: modularità, leggibilità, separazione delle responsabilità.
- Sicurezza e robustezza (gestione errori, guardrails, nessun segreto hardcoded).
- Riproducibilità: il progetto deve poter essere avviato da zero con la documentazione fornita.

Implementazione AI – 8 pt

- La categoria centrale del corso.
- Prompt engineering difensivo, versionato e documentato.
- Tecniche AI (RAG o Fine-Tuning) con pipeline dati di qualità.
- Consapevolezza nella scelta del modello.
- Uso sicuro dei tool con validazione I/O.

Valutazione e Critica – 8 pt

- Maturità ingegneristica: relazione tecnica chiara, valutazione strutturata, onestà sui limiti del sistema (costi, latenza, edge case).

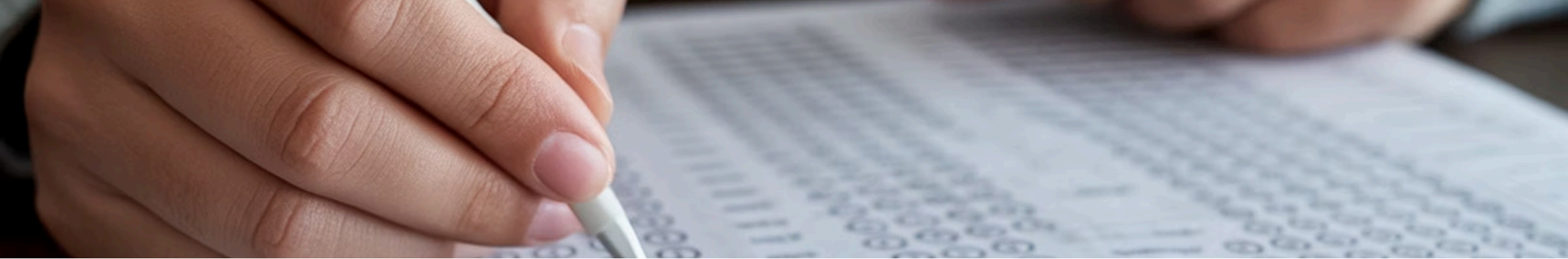
UX e Funzionalità – 6 pt

- L'app risolve il problema dichiarato con un flusso coerente.
- Dati presentati in modo comprensibile (Agentic UI).
- Resilienza: l'app non crasha e degrada con grazia.
- Interfacce pre-esistenti (Agno Playground, Streamlit, Gradio) sono accettabili.

📋 **Contributi Individuali:** Per i gruppi (max 3 persone) è obbligatoria una descrizione chiara di chi ha contribuito a cosa.

📋 **Bonus (+2 pt):** +1 per complessità tecnica (Multi-Agent, Custom MCP Server, Human-in-the-loop, orchestrazione complessa) e +1 per originalità (idea innovativa, applicazione non banale).

📋 **Uso di AI:** È consentito l'uso di assistenza AI nello sviluppo del progetto. Tuttavia, l'incapacità di spiegare il funzionamento del proprio codice durante la discussione orale impatterà negativamente sulla valutazione finale.



Prova Scritta

Il test verifica la comprensione profonda, non la memoria. Ecco due esempi di domande tipo che potreste trovare.

1

Domanda:

Perché l'architettura Transformer standard ha una complessità quadratica $O(L^2)$ rispetto alla lunghezza della sequenza?

- A) A causa del numero di layer nel Decoder.
- **B) Perché ogni token deve calcolare l'attenzione rispetto a tutti gli altri token (matrice $L \times L$).**
- C) Perché i vettori di Embedding sono troppo grandi.

Risposta Corretta: B. (Verifica la comprensione del meccanismo di Self-Attention).

2

Domanda:

Devi creare un assistente per un manuale tecnico di 5.000 pagine che cambia ogni settimana. Quale approccio è più efficiente ed economico?

- A) Fine-Tuning del modello ogni settimana con i nuovi dati.
- B) Inserire tutto il manuale nel System Prompt (Long Context).
- **C) Architettura RAG (Retrieval Augmented Generation) con Vector DB.**

Risposta Corretta: C. (Verifica la capacità di scegliere l'architettura giusta in base a costi e aggiornabilità).

📌 Le domande all'esame saranno 30, a risposta chiusa, basate su scenari simili.



Voto Finale

La valutazione finale del corso segue un algoritmo preciso che combina il punteggio ottenuto nel progetto pratico e nella prova scritta. È fondamentale superare entrambe le componenti per ottenere l'idoneità.

Voto Finale

$$V(Finale) = \frac{2}{3}V(Progetto) + \frac{1}{3}V(Scritto)$$

Il voto finale viene arrotondato all'intero più vicino.

Condizioni Necessarie

Voto Progetto $\geq 18/30$

Voto Scritto $\geq 18/30$

Se una delle due prove è **insufficiente**, l'esame non è superato, indipendentemente dal punteggio nell'altra.

Criterio per assegnare la Lode

La Lode viene assegnata automaticamente se la media calcolata **supera 30.5**.

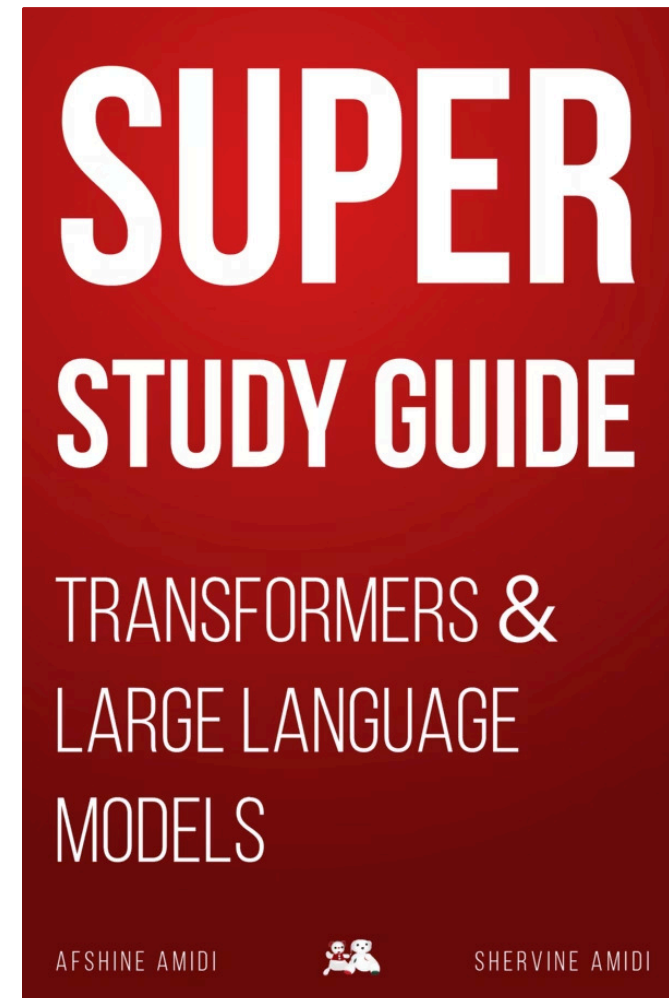
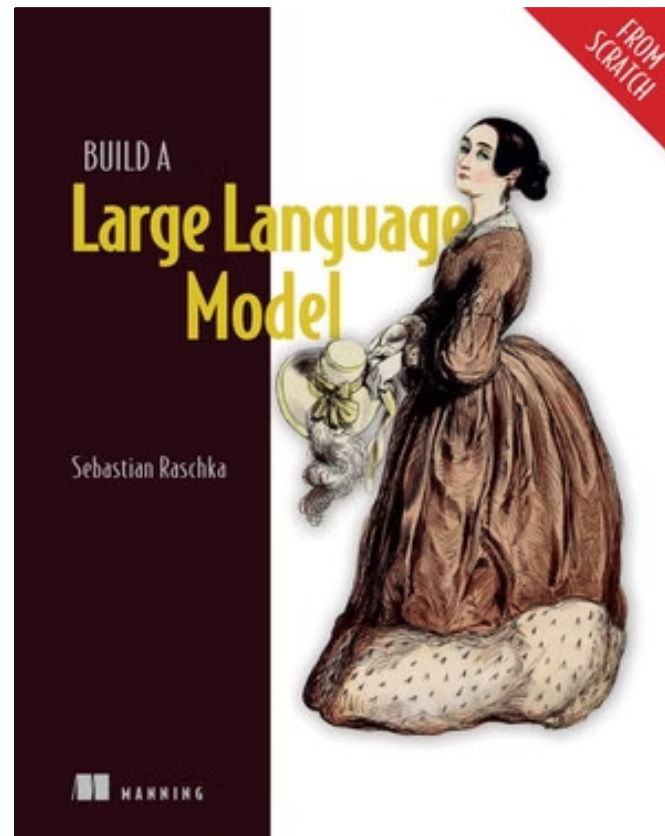
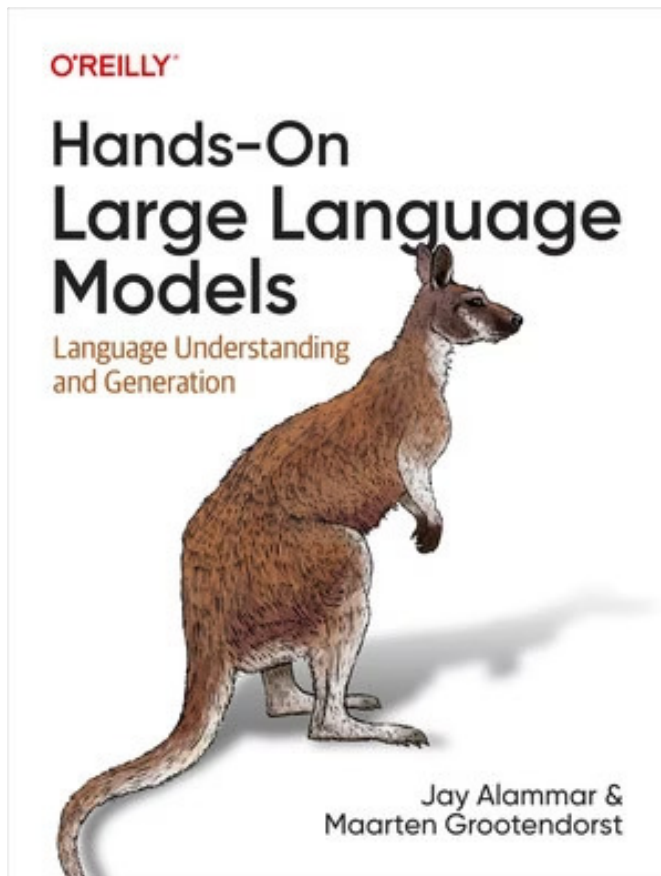
Questo è possibile grazie ai **2 punti bonus** previsti per il progetto.

Esempio: Progetto 32 (30+2) + Scritto 28 = Media 30.6

→ 30 e Lode.

Un 30/30 puro in entrambe le prove (senza bonus) dà una media di 30 (senza Lode).

Libri di testo



Domande?

